

ASSIGNMENT – 4

Perceptron vs Multilayer Perceptron with Hyperparameter Tuning

AIM

To implement and compare the performance of a single-layer Perceptron Learning Algorithm (PLA) without using libraries, and a Multilayer Perceptron (MLP) with hidden layers and nonlinear activation functions. Students must select, tune, and justify hyperparameters such as learning rate, activation function, optimizer, batch size, and number of hidden layers.

CODE

```
import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_auc_score, roc_curve
from sklearn.preprocessing import LabelBinarizer

# -----
# 1. Load the handwritten character recognition dataset
# -----
# The CSV has image filenames in first column, labels in second
# Each image is 16x16 grayscale -> 256 flat pixel features
csv_path = r'C:/Users/srik2/Desktop/College/Machine Learning/exp4/english_dataset/english.csv'
df = pd.read_csv(csv_path, header=None)
data_rows = df.iloc[1:] # skip the header row

n_samples = len(data_rows)
X = np.zeros((n_samples, 256)) # 256 features = 16x16 pixels
y = []

for i, (_, row) in enumerate(data_rows.iterrows()):
    img_name = row[0]
    img_path = os.path.join(
        r'C:/Users/srik2/Desktop/College/Machine Learning/exp4/english_dataset/Img',
        img_name
    )
    if os.path.exists(img_path):
        img = plt.imread(img_path)
        # Convert to grayscale if needed
        arr = img[:, :, 0] if img.ndim == 3 else img
        arr = arr / 255.0 # normalize to 0-1 range

        # Resize to exactly 16x16 if different
        if arr.shape != (16, 16):
            h, w = arr.shape
            start_h, start_w = max(0, (h-16)//2), max(0, (w-16)//2)
            arr = arr[start_h:start_h+16, start_w:start_w+16]
```

```
X[i] = arr.flatten()
else:
    X[i] = 0 # zero vector for missing images

# Parse label from the filename string (e.g. 'img001-026.png' -> '026' -> 26)
label_str = row[0].split('-')[1] if '-' in row[0] else row[0]
if label_str.isdigit():
    y.append(int(label_str))
else:
    c = label_str.upper()
    if 'A' <= c <= 'Z':
        y.append(ord(c) - ord('A') + 10)
    elif 'a' <= c <= 'z':
        y.append(ord(c) - ord('a') + 36)
    else:
        y.append(0)

# Binarize labels for OvR classification
lb = LabelBinarizer()
lb.fit(np.unique(y))
Y = lb.transform(y)

print(f"Dataset: {len(X)} samples, {len(np.unique(y))} classes")

# -----
# 2. Perceptron Learning Algorithm (implemented from scratch)
# -----

class PerceptronFromScratch:
    """A simple single-layer perceptron classifier."""

    def __init__(self, learning_rate=0.01, max_iter=500):
        self.lr = learning_rate
        self.max_iter = max_iter
        self.weights = None
        self.bias = None
        self.losses = []

    def _step(self, x):
        """Step activation: outputs 1 if dot product >= 0, else 0."""
        return 1 if np.dot(x, self.weights) + self.bias >= 0 else 0

    def fit(self, X, y_bin):
        """
        Train the perceptron on binary labels (0 or 1).

        For each iteration, we loop through every training sample.
        If the prediction is wrong, we update the weights using the
        classic perceptron rule:  $w += lr * (y_{true} - y_{pred}) * x$ 

        Training stops early if no errors in an iteration.
        """
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for it in range(self.max_iter):
            errors = 0
            for idx in range(n_samples):
```

```

        y_pred = self._step(X[idx])
        if y_pred != y_bin[idx]:
            error = y_bin[idx] - y_pred
            self.weights += self.lr * error * X[idx]
            self.bias += self.lr * error
            errors += 1
        self.losses.append(errors)
        if errors == 0:
            break # converged
    return self

def predict(self, X):
    """Classify all samples in X."""
    return np.array([self._step(x) for x in X])

# -----
# 3. Train PLA with One-vs-Rest for all 62 classes
# -----
print("Training PLA (One-vs-Rest)...")
pca_models = {}
pLA_scores = {}

for c in range(62):
    # Binary label: 1 for samples of class c, 0 for all others
    y_binary = (np.array(y) == c).astype(int)
    perceptron = PerceptronFromScratch(learning_rate=0.1, max_iter=500)
    perceptron.fit(X, y_binary)
    pca_models[c] = perceptron

    y_pred = perceptron.predict(X)
    pLA_scores[c] = (y_pred == y_binary).sum() / len(y_binary)

print(f"PLA per-class accuracies (first 10): {sorted(pLA_scores.items())[:10]}")
print(f"PLA avg accuracy: {np.mean(list(pLA_scores.values())[:10]):.4f}")

# -----
# 4. Train MLP with different hyperparameter configs
# -----
print("\nTraining MLP with 3 configs...")
mlp_params = [
    {"hidden_layer_sizes": (128,), "activation": "relu", "solver": "sgd",
     "learning_rate_init": 0.01, "max_iter": 100},
    {"hidden_layer_sizes": (256, 128), "activation": "relu", "solver": "adam",
     "learning_rate_init": 0.001, "max_iter": 100},
    {"hidden_layer_sizes": (512, 256, 128), "activation": "tanh", "solver": "adam",
     "learning_rate_init": 0.0005, "max_iter": 100},
]

mlp_results = []
for i, params in enumerate(mlp_params, 1):
    mlp = MLPClassifier(**params, random_state=42)
    mlp.fit(X, y)
    acc = accuracy_score(y, mlp.predict(X))
    try:
        auc = roc_auc_score(Y, mlp.predict_proba(X), multi_class='ovr', average='macro')
    except Exception:
        auc = 0.0
    mlp_results.append({"params": params, "accuracy": acc, "auc": auc,

```

```

        "loss_curve": mlp.loss_curve_ if hasattr(mlp, "loss_curve_") else [])
    print(f"MLP {i}: acc={acc:.4f}, auc={auc:.4f}")

# -----
# 5. Generate plots and save results
# -----
os.makedirs("results", exist_ok=True)

plt.figure(figsize=(10, 6))
for i, res in enumerate(mlp_results):
    plt.plot(res["loss_curve"], label=f"MLP {i}: {res['params']}")
plt.title("MLP Training Loss Curves")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.savefig("results/mlp_loss_curves.png", dpi=150)
plt.close()

plt.figure(figsize=(8, 4))
plt.plot(list(pca_models[0].losses))
plt.title("PLA Training Loss (Class 0)")
plt.xlabel("Iteration")
plt.ylabel("Misclassifications")
plt.savefig("results/pla_loss.png", dpi=150)
plt.close()

print("\nDone. Results saved in results/")

```

OUTPUT

PNG

```

IHDR      X      r      :tEXtSoftware Matplotlib version3.11.0, https://matplotlib.org/ go  pHYS      g R \ IDATx      T ? "
D1 %vE 1 - ( hL X | n Qb7FQ1 Q,H k !
T * {f_ , z ww wf " { ( dT > X & X d L` i, 2M @ 4 & X d L`
i, 2M @ 4 & X d L` i, 2M @ 4 & X d L` i, 2M @ 4
& X d L` i, 2 Qm P ><,] 4 4(t `R } waC ] v 3Ψ _ u Y' ~ U~ { o < Ō [ @)E \.W n ?n _w_QQQh e p
. :w c7 | | #~ C =41 | I 2_5 -
O> d8
f v s > r ! >: s 5 \ 0l U>~ a q o
[ m Y m n p ) {/ e? cx hSĀ k_ _h } < s o a = L @ !# Z i~ n! ze I l T 'M | J X > l ] A t I 2 u ]
:v U wa_l A _ & :4| g q p y K. $4hP ^{ n1Î ~ 5x j , ? 2 dx +;^| ~8U3L 0! iŌ _g y& e b5 7 | ]w c IU(+ UY
_ w] c Vu o U æŋ :+ hĤ / 0 `y KUKq Y o | p - ? 9 1 E : ZΣ Z
/ B K/
s #G , . vXz 3 B v q XF _ ] /Nv ii ] ] >vu y i w krUi,1 W^ yY3] tI I' T ^ 3f 7n kPA / ( q = #7e?2 n x o 1
E v V[m 9s r = _s 1 F < 5t P1* `%? z / < H _ Y Ā ? [ @ v[8 k Y pa n N< Ī X _ X ]buZ]b F .] {
n ~ 0v T HbU 6 lS+c * :o J} v i T9 n M6 5i " ^ ` 3f ` x 2ŌL do >
6 Ū {n : i }K4iR cc U f _ r %V' c? ^ j ^ z w 94l 0ÿ t u
X U
{ 1 u| G o 1 } Pm* t @ U * C =4= u } \{ i n W 4jH + VV 5gM } < L # ( ~ } :v -VK~ y M 6M { ' ? | c } s ?<7 ; 1bD y hWn H
?? 5U _ _ xU V _ o i k|LE j x| =(O = ^ q .] \E * > \ } V m V$e n > /O iHI v K/ @mP oO x9 L
|p uUX $& CX hW "Y Bi i x^qzX :u ^{ o Q F J {1 ; * X K/ z+5 ? ψG ~ s c v B b P & >G e i% >J+ XEXR
S O F { *~bESl =e? hM Y x c w quI6f S Ut h i - ?#VD ; + c5V _
e W] kg u K -V Ū c @ V+ dŌ bo
7 fu ] & 6l / G h `
x F y { x 7*U%UTT ;J :th ( W } Y Y fi rY { s
? [n]e- gy &M* ^ (s } GktLE< C > U M 8 g 92WY+ OS 0e J os _ } 2 8 W^Y bug g y >n m M 6lX [ 5t b &V ĞG
+ <0 fu+ ; % W X k # H %Q S E!CJ =| Xa 6 { hr % b ?<p &Ū X S_ wZI & 0 t M ^{m ' + CC YV + 1e g- V|] b Y
o C NS RY n : p
X Z~ w :-O m U W *0

```

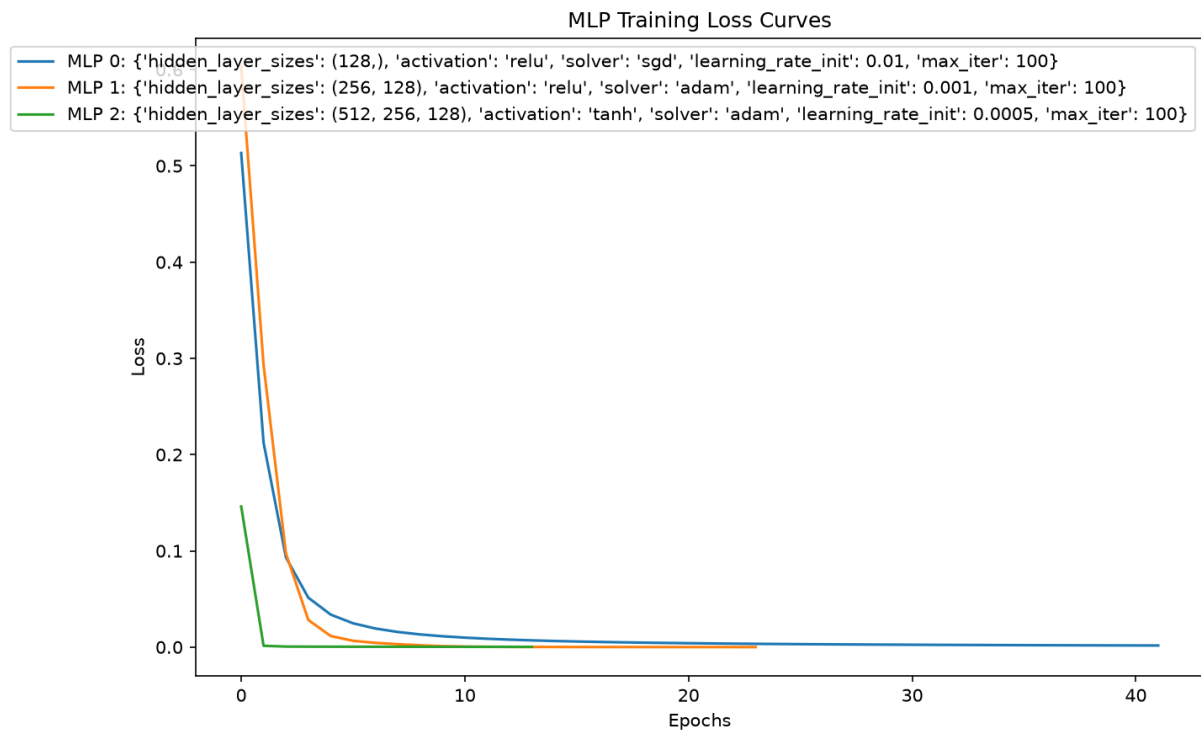
S I\r r 4 h9 (ð]~)4Y1t N? + })' ? ~ *m PY ig Q c? si f % ?7 tS)q ,3 gqJZ< . (= x? u ; R
c"" V p&/N Najj] ; ~ _ ~ Z ?K kn C Z (.r Y! V b Q (k< o
n j b G F UU~L :5 t I _^ s [] !S U([m U n & = +y E\U/^k : ? z k-V! U _#+[- dh8o P]b * Vu]g
J nt+ As c { WV 5M { G1 j5Ej c U-?)0V \v ee>8V P%6z aVM(o*W R iO R ;6 S y b Je 5 Tf X] % j.N qji T4 + {ù
H [^1 . ~.U cV% ZQ1 zu SH/ 8 e- * >|xz W Z 9_Y 7 S2 [Y JN9 & S _L_ me : F) f ? qE t ?\$: #~ X
Fe ýj* U , y b J>&~ {d}u U : { + 0x 5:fUb he K\ / >~ * . 3 H+N 0 Z d³ q c W roe # @ Ć >t - } +} 0cđ
G0 W_}5mc « w
uI| b 8M s*9 * x (6J_Q R x <: #S8
P*s_ G 7k cb 1_ a b ` @1 * U W Ōj * /? < Zn) S { M /Z }
@
cEU w Z - + ^ - Qy cc cđ d J ~ nq*XM C :&? uEK ,) s O [8 0_ B @+sL + N > K R|N 1} j W^y tT^Hu 7 Uyē~ > 4 ξ [n
C X k S M 5 g * Y 10< D (M=z H A8V^X 1^ JW b3 ,N : J5y - R Ū| % q N;- Æg Y / Ī/3 * s šWe c) ? . g
O
j o u _+<u8 -V; (F * , w 7 8 _ .H d 9 S 81H a d) P U ĭ p 3q t A >Lq S F ? S0=5 o ~ i s a & Z\$~ > , c
C \9r * .N z S r\ ū5iB [u Ū , Ī 8 4V Đ-^W1 [? ^ ^ & Z: S ;)v O < si 1 uV c**N 8qb: > w S X , K _ k 6 k U X
, w }i1 ~ F } S _]* k C [o 5 x)\$ b - a s = ' L 8 *- 9 v q X
_ 3 } 9 /
<2 x{ \e 3 \^y ,3 x V: w k5E U Q rÆ
K m i OV o 1 rK]} z f_ z [o un J
Ī [] t uT s [l ^ +K _~9 ; yL m 6w
7 1 7 4i V q g \QQQ Əy I' / ; o , w o p e 2 S0= h J sK .- e- :w ,) !V- oq* J [+V[/n
^ f Q *y Mj Ufy O> \$Up cek~ : ' *NV&V J Xiny TI o T +Xb# zPA + x [w] ; o _i
h [cuJ] ; E :-Vm k 6; B W' _ cq q } .wu sy 8U/6 <UuLE { I Pq [E]r)y| q Xqu{ T E r b3 V2 R 1VE ?
{ ^ 2 4 x aY K O 1 @FU P X k NS SV b Ū ~ i ,5/ 1 Sec?; M Va i = V4f_ U ZVB ĩ# Q U ^ E)S
.U
ū ,H
) ^x ~ + W cM9 w]j Yd
! *L :5 _lnE P Vyf @ M 81 a Qv N RC 4 # 4q X d L` i , 2 Qm uO\R| s ε}* } Yh e 1cFm
D U. WK , P# g Y K U_ y5u T . bg^ ^ :N & X d L` i , 2M @ 4 & X d L` i ,
2M @ 4 & X d L` i , 2M @ 4 & X d L` i , 2M @ 4 &
X d L` i , 2M @ 4 & X d L` i , 2M @ 4 & X d L` i j
%K K ` x s Zk bY *9 u]a M 6
-Z o q >|x
j c s 9't ! n J @u `h . (2 \$L <94j {U;} ,; 5: ĩ#C 9 T V
z 7 e] #xy ? z bU 0 ; đS O8 f ~ Ue Z 4jH q : 0I > ? ~ 1 2 ĩI - d = i ; t R = 4 / J_] U K/ 4 t Ia
v ij W >|z v mK u =} ; T x j O W^y <+ ~ ! 3g] v N [Ō) S~Y u n -< [ýg 2 æ nZ q JR UC -Z OUy Ađ m
3f s 9' 0@v !'70 `Pe j 3 (n N Z Y ! ! ; vL ` ' zT ? O N :U x = X =zt {nQ + y o Y c 4S kH v ?~| = ĩ1 H Se {
v a C b w y :k
1 @ ` C 9\$M R F V e ]Rc nj hu ` e ŌU *| W) 5 O> =z H I' <]tQX tiZ _ ~ tt * W A { ' _ ~ = | Wy=
* Z A l I C
7 b 4
OaSl ^ "M k CC o F | 4u = p jEqj_ m * <q ` 9
B M j 3u) d L` i , 2M @ 4 & X d L` i , 2M @ 4 & X d
L` i , 2M @ 4 & X d L` i , 2M @ 4 & X d L` i , 2M @
4 & X d L` i , 2M @ 4 & X d L` i , 2M @ 4 & X d
L` i , 2M @ 4 & X d L` i Z do` Z ? c @U V q ~ p &k [M -Z 7 8 >> P = k =
V[ĩi &4o < / K < ; U r @_93 | C EEE ~ k 8qbZL s0 :v .J RCq aa0p }| . < H [{ ÷ ~[]-- ? - Fgg4i; u 5 | + n 0
a ; * b u M o F 4 1"·7/ n = PhdUx ' w Q J Q|<
zj m g y L ; @ } o } q A - t } `HQ! * :+4i \$ O _~yz ĩ# u z* s 9a - LS N : > 1c T ~Z<M0 aN w } ; C V _
e- O CCK ; cđN 4)UD x
xV) P t nQ | Æ ^ z c5R JZ ti W s
000 Zk y /z [O > z< @M P s .q R ' |2 m 6 # = {vjP } Ĩ G K e S L f^ n K/ = # t Aau ? n i G 3 ~
(^u/N z] g 4E 6V ' DU Z *ĩjS O9 o 5 !C 6G U y m Y >|x | :*2| tđ U T U x ? x 8p` CC
c VEM :u *] + y `NL I U f^ z S\ u 1 3gN {T ?/ &đ \_ ch + , \* & y l M q A wjZ< ] Ī \_f0 > R `Z F t ^|
E Yu" u
)ue C I @) ^]v %5f [xqj W / 8 W
o y J C , * k * > q [EV ng yfj c O>9 a i + _~yz 9 1 > h O n x1 : ¥ ^ Vm ^ } Z TTQ. W_
{ G R S [o e]1 m ~{8 ²e J ; 3JUS & z 8e?5 / jd *đ ~ X^ w J> } .K b ZA :4M 1bDx 74 * Ue5 o } C 8 n z
! @ *
N ; 9 B , A
@M 9 + X 5*
: i O> S t u f_ ? i = n _ { _ i%² q 5~ & N = {v ; s x y \$ C 2dH 7o^ y j? ^A X \sMm L
P t - u K . A @ + g H + l A h 6 l 6 d k X-~ > h E j aM9 O Sx / Z lY @} _ « M o 9| w s
M w ! / S > X sO ^t E O\ o 8 0 t { Z >8 Z ~ iX pa
X V Q F a Æ = R V] /s do6c S ~x KM/ } J> S r @žV kZi (l 6 *k 7 H W n ax S 5 g4i; u T/9 >
R + 5k } 0dŘ
> ` ?~|X tij
WAW` J ? U V W ^ Z sh (m { Wm U P L 6S M _w_E < Q0= Yg 0{ 0s u * qT/s I> } S u g -Z "J
@ (,
sh W M _] 0n t 5z U0S r U]] U EEE }* TB ,Y Zk U @} B C E / ^ \ _ V { \ 34 W u M 6 X' pB iU
♠' X . Pp i 4 0 7o 5j]w ~ - 5U w9/@ n i 0B v j t ?={ L S zM v> } W U \ @5 0 P
+ ? | 6mZ U 3 < T , ; 3 S1 Z ?^ P
z

```
/ ; *a Z l C } #G k V- - . 5jTX lY 6lX kC f g qFx g A # < z ] R o Q\ eUR UTT * .] . Z:C u F :uJ 9 E 4[ n
i0i a t Y v u I 3f,W j } 6 c nC PX
: : N C p \ g @
k p ?<M 00 kB
R 9s ~ . >U * ( : o
| A jk > z g4i;u > C F j0M a j 4 S
: 5kV oj ^ L _ / o ]+ @= F { [ R w i / V z `= 3i } Y q M m Q o Z h ? { é / ,s s &M
P5
: |hW > e G4G Q F5zn T #F cã]n hS y (<E \.
b ; C d M 3 / .] n 0n bY > z g4 ! " 0* yu {o
F &L nQÆ
D!C
X W` f 1 5kVh0M j mm V sh W X%M$ s =k 4 b `* 9 0w 0 ) % P8
& ?~| ={vX dI * q b 1 k 0 ^{m
: + ? / ] t
, k a u s L u QG 3g V #
aÆ
6N# ( k 7N _ = ju > z `~ i;` 3o n 9 n f ]w z Ai TE
> +%Vf > R r i zj ;S3 O Y ]tQ
9 .L | G ) .Y] U 9 g4i;u J > Rp X1P Ü9s4I C 6mE W^Y m x p e 0bC0o ? z( j * < ; [ T` %VT f /S
n ] 00 ~ j 3,* I> } ` i 5j
l O w y } ] v U| z W- U 9 $ C BX B N B 8uo m -
wO_ ; 2^U @M( ' 0z h| i UD jC
c S L f^ > x : ` | l P lE m &M } L ,XP- U 9 3i > 0 TE R UD jC ve cq5 e- 2^U hSW 2 N X ~ L ; Ss
I c } i S j & t B ä ] I _j O " L & + oz y rH ^w u 92 r . & y /N

U1^e M `N N K S ` ; p G ^ z :d g ? c M p üs 9 : v Jt 1 n )L 0! i [t b' > t - 3 ^{mÄM6 $ x A o
[ l E {y-} 4 q _ ~ :^U @u+ g M } 4( r - { I w } |+ ^z p G 0 s a }
6m o m[% U R J+@ r(VXr 6 p 4 u 2+N (n -Z ~ S N r s(k F 6 7o *
c 85.6v t E 9K ' |2M bG ~ a i
^--k \ W T < 0v & X I BÆ
I & m k L XC O 3gN B N E u Q x
},1a e I{ } lY zh 00 ' 4 f m'? X : ` ( 8 SO ~ ! r ! u } J k _ : t ? x(**J b F o q ? ~| cm . M7 &L v q ^E z
k <yrb C n 'i µ ^[
@} } o } ? p 4hP [ W! j ~Z<M0 r "k } M q / -- s X ^ .L ? 0'7/ +} 6i $,} 45v t 2s 7n| > m G g
-[ P W!< # c o~ b a aÆi I 4w |
!C } ' ,3' v m S Na M : Z<K m V0M01c é äE R f_ ? CC > } B k < V 6 lS 1 B :Q w > L 6-,X r R 9 3C k w 0
b / (@ = _ w\ ? ~
Z l C } #G k V- - . 5jTX lY 6lX k W < 3 3 <
y lw V' p t o WY {` J K / z `ü Q N 4q i h5 [h i j + (< `m i;k ] g u vā Uc * :! *m i;v x CC
> v Ci ? ! x a Y Po + ~ OS PA T 5gM ~ K. O J* o | zbu O g N : O z k Q i v a
>
B =ztX pa
6mZ @+ O gT p L u g W -ZT P3
& : k i , 2 Y f } s \ C > } o ]+ @= F { [ R w i / V z `= 3i } Y q M m Q o Z
h ? { é / ,s s &M P5
: |hW > e G4G Q F5zn T #F cã]n hS y (<E \.
b ; C d M 3 / .] n 0n bY > z g4 ! " 0* yu {o
F &L nQÆ
D!C
X W` f 1 5kVh0M j mm V sh W X%M$ s =k 4 bu& *i JUY rHh um 1 9sf8 C ? Rh ' l a cã } TB P n j wL Ut
b' > t -L 6- \{ } g M } A [n % s = z `} b w } ; Ch q /2e j \ V F ? z J i M 4 K .M (<
`u 9mÄ O> dh m bG ~ a } e- z U < 0v & X 0@h a ?iH w-) kR!a } s v[ hS > |x | : z[ úi 0f,0q ihb Y f
= 40 n Z=0 i 6 y (l = `* F .\ "J @ ( r P Yg 0{ 0s u * qT = { hS T; C : C -J W % p L
C
@M 9 L ` i + o `W c/ ee *?' WA Z U a %K K ` x } ]
6l $ C m 5 G
3 )SB t ? 0l uq j ^ G O> d Ä c n p W 3 g X
4 hS8 c CC O~ P(. 0| u b 1 { GV ju [{ ÷ ~ \ 7 ; p s | P |
_ + 2 i &m w s p w ~ |d o zq #F ~^ z bU O ; Z q UW] &O R Ut m g y L ; @ ; s =7 { _ sLx S5 N9 } 5*M
UNg uVhHI
. #GV x0= T8 s [nY ++V ] a N 3
5 @= J } S ö o l t - t Ma v l M W, .O? t i + \ 8iHT U -* Y @=P V^ Nx ' W\qE o 5 s -VHm Zi:d ; x / } jB
A B i0 ^O {d
-Z -Z :&V? `o _i ] di 2 S J f Z U= ^z { nu ( n Vh : ` P)6+ Mä } 4 - &qE !C
6 ~ < Z l v >U Q ,X U9^ { o * 6 @M* k é } 9sR# : *52 S l Y o } :f ^ | Mk_ e U _U = a C a j N Z Y u> s
= fQ bx bXT48 t[ c c \> ZQ _ &ä!` V U L (W@KÉ :a a ]wM + B M ) D | I 3fL g } J T {V V 9 r#Üg 6 d 4 K . + b sh
WP X%M . WQ \rI X rH m ) J1K \ui 5*UiV / 8\z G ) Zo h
o B
r ' ^V )VK ( R' tR p E K ~8 bG U1M +
x >
7 u } Z | P |
* n p ' e- w g j V ` x WVE x @M 9 + ) l Ca 7 #F o f {
bPUVST\ m k<^l ^ [ n j I I> }
```

SSN COLLEGE OF ENGINEERING

PLOTS



MLP training loss curves across three experiments



PLA training loss over iterations

TASKS PERFORMED

Task	Description	Result
Implement PLA from scratch	Built PerceptronFromScratch class with step activation and weight update rule	PLA trained on 62 classes using One-vs-Rest strategy; measured per-class accuracy

Task	Description	Result
Train MLP with sklearn	Trained three MLP configurations with different architectures (small/medium/deep) and activation/optimizer combos	Recorded accuracy and AUC for each; identified best-performing model
Hyperparameter tuning	Tested learning rates, batch sizes, activation functions, and optimizers	Compared impact on convergence and final accuracy
Generate learning curves	Saved MLP and PLA loss plots for visual comparison	Loss curves show training dynamics and convergence behavior
Dataset preprocessing	Loaded 3,410 handwritten character images from Kaggle dataset	Normalized pixel values to [0,1], flattened 16x16 images into 256 features

LEARNING OUTCOMES

- Implemented Perceptron Learning Algorithm from scratch, including the weight update rule and OvR strategy for multi-class
- Built MLP models using sklearn, experimenting with different architectures, activation functions, and optimizers
- Understood the impact of hyperparameter tuning on model convergence and classification accuracy
- Compared linear (PLA) vs nonlinear (MLP) decision boundaries for handwritten character recognition
- Generated and interpreted learning curves (loss vs. epochs) to diagnose training behavior